DOCUMENTO OFICIAL DE INGENIERÍA TDD

Catálogo Exhaustivo y Detallado de Pruebas Unitarias, Integración y E2E

Medicent Backend V3 — Matriz Individualizada (29 Implementadas + 11 Planificadas = 40 Casos)

EQUIPO DE DESARROLLO	TOTAL DE CASOS	IMPLEMENTADAS (OK)	PLANIFICADAS
Andrés & Cristian Beltrán	40 Pruebas Individuales	**29 en Verde (100% Passing)**	**11 en Roadmap**

¿Cómo está estructurado cada caso individual en este catálogo?
Siguiendo el estándar universal de ingeniería TDD, cada prueba cuenta con su ficha técnica autónoma bajo el patrón **AAA (Arrange / Act / Assert)**. Esto garantiza que cualquier prueba que comparta archivo con otra sea evaluada, ejecutada y auditada como una unidad independiente y verificable.

1. Resumen de la Matriz de Pruebas por Niveles de Arquitectura

Nivel	Dominio / Alcance	Implementadas	Planificadas	Total	Archivos Asociados
Nivel 1: Unitarias	Lógica de negocio pura (Auth, tomas, modelos en memoria)	**5 casos**	5 casos	**10 casos**	tests/unit/test_*.py
Nivel 2: Integración	Servicios ORM, Base SQLite, Catálogo, Prescripciones y Endpoints REST	**24 casos**	1 caso	**25 casos**	tests/integration/test_*.py
Nivel 3: End-to-End	Flujos HTTP completos (Healthcheck, Auth, Prescripción y Auditoría)	0 casos	5 casos	**5 casos**	tests/e2e/test_flujo_clinico_e2e.py
TOTAL GENERAL	**Suite Integral del Backend**	**29 CASOS (OK)**	**11 CASOS**	**40 PRUEBAS**	**Cobertura Total**

2. Catálogo Detallado de Pruebas Unitarias (Nivel 1 — Funciones Aisladas)

U1: test_registro_usuario_exitoso

[U1] IMPLEMENTADA

Archivo: tests/unit/test_auth_service.py | **Nivel:** Unitaria
Objetivo: Validar que auth_service.registrar_usuario(...) registre un nuevo usuario, cifre la clave con Bcrypt y devuelva ID primario.
PREPARAR (ARRANGE): Parámetros: nombre="Juan Perez", correo="juan@test.com", contrasena="secret123", rol="paciente".
ACTUAR (ACT): Invocar auth_service.registrar_usuario(nombre, correo, contrasena, rol).
VERIFICAR (ASSERT): usuario is not None, usuario.id asignado, hash Bcrypt inicia con $2b$ y no es texto plano.

U2: test_registro_correo_duplicado

[U2] IMPLEMENTADA

Archivo: tests/unit/test_auth_service.py | **Nivel:** Unitaria
Objetivo: Asegurar que si un correo electrónico ya existe en la base de datos, el servicio rechace la operación retornando None.
PREPARAR (ARRANGE): Usuario registrado previamente con correo "duplicado@test.com".
ACTUAR (ACT): Intentar registrar un segundo usuario con idéntico correo electrónico.
VERIFICAR (ASSERT): El resultado es None y no se generan registros duplicados ni excepciones no controladas.

U3: test_login_exitoso [U3] IMPLEMENTADA

Archivo: tests/unit/test_auth_service.py | **Nivel:** Unitaria
Objetivo: Validar que un usuario con credenciales correctas reciba un diccionario de autenticación con Token JWT y claims de rol.
PREPARAR (ARRANGE): Usuario registrado en BD con correo="login@test.com", contrasena="password123" y rol="cuidador".
ACTUAR (ACT): Invocar auth_service.iniciar_sesion("login@test.com", "password123").
VERIFICAR (ASSERT): Retorna diccionario con clave "token", usuario y rol "cuidador".

U4: test_login_contrasena_incorrecta [U4] IMPLEMENTADA

Archivo: tests/unit/test_auth_service.py | **Nivel:** Unitaria
Objetivo: Comprobar que si la contraseña provista no coincide con el hash almacenado, el inicio de sesión sea denegado.
PREPARAR (ARRANGE): Usuario registrado con clave "correcta123".
ACTUAR (ACT): Invocar auth_service.iniciar_sesion("login@test.com", "incorrecta999").
VERIFICAR (ASSERT): El resultado retornado es None y ningún token JWT es generado.

U5: test_algoritmo_calculo_tomas_intervalo_regular [U5] IMPLEMENTADA

Archivo: tests/unit/test_calculo_tomas.py | **Nivel:** Unitaria
Objetivo: Validar la exactitud matemática del motor de calendarización: 2 días (48h) cada 8h genera exactamente 6 timestamps regulares.
PREPARAR (ARRANGE): Rango temporal: inicio 2026-11-01, fin 2026-11-02, hora 08:00:00, frecuencia 8 horas.
ACTUAR (ACT): Invocar tratamiento_service.calcular_horarios_tomas(...).
VERIFICAR (ASSERT): len(horarios) == 6, primer horario a las 08:00:00 y diferencia temporal constante de 8 horas entre tomas.

U6: test_validacion_algoritmo_tomas_frecuencia_invalida [U6] PLANIFICADA

Archivo: tests/unit/test_calculo_tomas.py | **Nivel:** Unitaria
Objetivo: Garantizar que el motor matemático rechace frecuencias <= 0 evitando bucles infinitos.
PREPARAR (ARRANGE): Fechas válidas con frecuencia_horas = 0 o frecuencia_horas = -4.
ACTUAR (ACT): Invocar calcular_horarios_tomas(...) dentro de pytest.raises(ValueError).
VERIFICAR (ASSERT): Se lanza ValueError indicando que la frecuencia en horas debe ser estrictamente mayor a 0.

U7: test_modelo_usuario_instanciacion_y_atributos [U7] PLANIFICADA

Archivo: tests/unit/test_modelos_unitarios.py | **Nivel:** Unitaria
Objetivo: Validar que la clase Usuario asigne sus propiedades en memoria y oculte el hash en to_dict().
PREPARAR (ARRANGE): Usuario(nombre="Carlos", correo="carlos@test.com", contrasena_hash="hash123", rol="cuidador").
ACTUAR (ACT): Instanciar y ejecutar to_dict().
VERIFICAR (ASSERT): Campos asignados y la clave contrasena_hash no se encuentra expuesta en el diccionario.

U8: test_modelo_medicamento_instanciacion [U8] PLANIFICADA

Archivo: tests/unit/test_modelos_unitarios.py | **Nivel:** Unitaria
Objetivo: Validar instanciación en memoria del modelo Medicamento con sus atributos farmacológicos.
PREPARAR (ARRANGE): Medicamento(nombre="Loratadina", descripcion="Antihistamínico", forma_farmaceutica="Comprimido").
ACTUAR (ACT): Ejecutar med.to_dict().
VERIFICAR (ASSERT): El diccionario contiene claves "nombre", "descripcion" y "forma_farmaceutica".

U9: test_modelo_tratamiento_instanciacion_y_defaults [U9] PLANIFICADA

Archivo: tests/unit/test_modelos_unitarios.py | **Nivel:** Unitaria
Objetivo: Comprobar la instanciación en memoria de Tratamiento antes de ser persistido en BD.
PREPARAR (ARRANGE): Parámetros válidos de prescripción en memoria.
ACTUAR (ACT): Instanciar Tratamiento(...).
VERIFICAR (ASSERT): Atributos correctamente inicializados y consistencia de tipos de datos.

U10: test_modelo_registro_toma_estado_inicial [U10] PLANIFICADA

Archivo: tests/unit/test_modelos_unitarios.py | **Nivel:** Unitaria
Objetivo: Confirmar que una instancia de RegistroToma inicie con estado "pendiente" y fecha de toma nula.
PREPARAR (ARRANGE): RegistroToma(tratamiento_id=1, fecha_hora_programada=datetime.now()).
ACTUAR (ACT): Evaluar propiedades de la entidad.
VERIFICAR (ASSERT): estado == "pendiente" y fecha_hora_toma is None.

3. Catálogo Detallado de Pruebas de Integración (Nivel 2 — Servicios, BD y Rutas)

I1: test_creacion_modelo_medicamento [I1] IMPLEMENTADA

Archivo: tests/integration/test_medicamento.py | **Nivel:** Integración
Objetivo: Persistir un medicamento en la base de datos de pruebas y verificar generación de su ID primario.
PREPARAR (ARRANGE): Medicamento("Paracetamol", "Analgésico", "Comprimido").
ACTUAR (ACT): db.session.add(med); db.session.commit().
VERIFICAR (ASSERT): med.id is not None y propiedades serializadas correctas.

I2: test_crear_medicamento_exitoso [I2] IMPLEMENTADA

Archivo: tests/integration/test_medicamento_service.py | **Nivel:** Integración
Objetivo: Validar que medicamento_service.crear_medicamento(...) persista en BD un fármaco válido.
PREPARAR (ARRANGE): nombre="Ibuprofeno", descripcion="Antiinflamatorio", forma="Cápsula".
ACTUAR (ACT): Invocar crear_medicamento(...).
VERIFICAR (ASSERT): Entidad persistida con ID asignado y atributos guardados.

I3: test_crear_medicamento_sin_nombre_lanza_error [I3] IMPLEMENTADA

Archivo: tests/integration/test_medicamento_service.py | **Nivel:** Integración
Objetivo: Asegurar que el servicio valide la presencia obligatoria del nombre del fármaco.
PREPARAR (ARRANGE): nombre="" (cadena vacía o espacios).
ACTUAR (ACT): Invocar crear_medicamento(...) dentro de assertRaises / pytest.raises.
VERIFICAR (ASSERT): Se lanza ValueError conteniendo la palabra "nombre".

I4: test_obtener_medicamentos_retorna_lista [I4] IMPLEMENTADA

Archivo: tests/integration/test_medicamento_service.py | **Nivel:** Integración
Objetivo: Crear medicamentos en BD y comprobar que el servicio retorne el catálogo completo.
PREPARAR (ARRANGE): Registrar en BD "Amoxicilina" y "Omeprazol".
ACTUAR (ACT): Invocar obtener_medicamentos().
VERIFICAR (ASSERT): Longitud de lista == 2 y presencia de ambos medicamentos en el resultado.

I5: test_obtener_medicamento_por_id_existente_e_inexistente [I5] IMPLEMENTADA

Archivo: tests/integration/test_medicamento_service.py | **Nivel:** Integración
Objetivo: Validar consulta por ID: retorna la entidad si existe o None si el ID no está registrado.
PREPARAR (ARRANGE): Fármaco registrado con ID conocido; ID inexistente 9999.
ACTUAR (ACT): Llamar obtener_medicamento_por_id(...) con ambos IDs.
VERIFICAR (ASSERT): Retorna el objeto válido para el ID existente y None para el ID 9999.

I6: test_crear_medicamento_endpoint_exitoso [I6] IMPLEMENTADA

Archivo: tests/integration/test_medicamento_routes.py | **Nivel:** Integración
Objetivo: Petición HTTP POST a /api/medicamentos debe responder 201 Created y JSON del fármaco creado.
PREPARAR (ARRANGE): Payload JSON con nombre, descripción y forma farmacéutica.
ACTUAR (ACT): client.post("/api/medicamentos", json=payload).
VERIFICAR (ASSERT): Status HTTP 201 y JSON con medicamento creado.

I7: test_crear_medicamento_sin_nombre_retorna_400 [I7] IMPLEMENTADA

Archivo: tests/integration/test_medicamento_routes.py | **Nivel:** Integración
Objetivo: Petición POST a /api/medicamentos sin nombre debe responder 400 Bad Request.
PREPARAR (ARRANGE): Payload JSON omitiendo la clave obligatoria "nombre".
ACTUAR (ACT): client.post("/api/medicamentos", json=payload).
VERIFICAR (ASSERT): Status HTTP 400 y mensaje de error en JSON.

I8: test_listar_medicamentos_endpoint [I8] IMPLEMENTADA

Archivo: tests/integration/test_medicamento_routes.py | **Nivel:** Integración
Objetivo: Petición GET a /api/medicamentos debe responder 200 OK y la lista del catálogo.
PREPARAR (ARRANGE): 2 medicamentos persistidos en la base de datos de pruebas.
ACTUAR (ACT): client.get("/api/medicamentos").
VERIFICAR (ASSERT): Status HTTP 200 y array JSON con 2 elementos.

I9: test_creacion_modelo_tratamiento [I9] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento.py | **Nivel:** Integración
Objetivo: Comprobar la integridad de claves foráneas entre Tratamiento, Usuario y Medicamento.
PREPARAR (ARRANGE): Paciente y medicamento persistidos en SQLite.
ACTUAR (ACT): Crear y persistir Tratamiento vinculando ambos identificadores.
VERIFICAR (ASSERT): tratamiento.id is not None y relaciones ORM resuelven correctamente.

I10: test_crear_tratamiento_exitoso_genera_tomas [I10] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento_service.py | **Nivel:** Integración
Objetivo: Al invocar crear_tratamiento(...), deben persistirse automáticamente en cascada todas sus tomas.
PREPARAR (ARRANGE): Prescripción de 2 días (48h) con frecuencia de 8 horas.
ACTUAR (ACT): Invocar tratamiento_service.crear_tratamiento(...).
VERIFICAR (ASSERT): Tratamiento con ID generado, len(tratamiento.tomas) == 6 y estado inicial "pendiente".

I11: test_crear_tratamiento_paciente_inexistente_lanza_error [I11] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento_service.py | **Nivel:** Integración
Objetivo: Rechazar la creación de tratamiento si el paciente_id no existe en la base de datos.
PREPARAR (ARRANGE): paciente_id = 9999 y medicamento existente.
ACTUAR (ACT): Llamar crear_tratamiento(...) dentro de bloque de excepción.
VERIFICAR (ASSERT): Se lanza ValueError indicando que el paciente no existe.

I12: test_crear_tratamiento_medicamento_inexistente_lanza_error [I12] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento_service.py | **Nivel:** Integración
Objetivo: Rechazar la creación de tratamiento si el medicamento_id no existe en el catálogo.
PREPARAR (ARRANGE): Paciente válido y medicamento_id = 9999.
ACTUAR (ACT): Invocar crear_tratamiento(...).
VERIFICAR (ASSERT): Se lanza ValueError conteniendo la palabra "medicamento".

I13: test_crear_tratamiento_fechas_o_frecuencia_invalidas [I13] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento_service.py | **Nivel:** Integración
Objetivo: Validar que fecha_fin no sea anterior a fecha_inicio y que la frecuencia sea mayor a cero.
PREPARAR (ARRANGE): Fechas invertidas o frecuencia = 0.
ACTUAR (ACT): Ejecutar el servicio creador para ambos casos inválidos.
VERIFICAR (ASSERT): En ambos casos se lanza ValueError bloqueando la persistencia.

I14: test_obtener_tratamientos_por_paciente [I14] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento_service.py | **Nivel:** Integración
Objetivo: Retornar únicamente los tratamientos del paciente consultado y lista vacía para pacientes sin datos.
PREPARAR (ARRANGE): Tratamiento para Paciente A; Paciente B sin prescripciones.
ACTUAR (ACT): Invocar obtener_tratamientos_por_paciente(...) para ambos IDs.
VERIFICAR (ASSERT): Longitud 1 para el paciente registrado y longitud 0 para el paciente sin datos.

I15: test_crear_tratamiento_endpoint_exitoso [I15] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento_routes.py | **Nivel:** Integración
Objetivo: POST a /api/tratamientos debe responder 201 Created y conteo de tomas autogeneradas.
PREPARAR (ARRANGE): Payload JSON completo con paciente_id, medicamento_id, dosis, frecuencia 8h y fechas.
ACTUAR (ACT): client.post("/api/tratamientos", json=payload).
VERIFICAR (ASSERT): Status HTTP 201, datos del tratamiento y total_tomas_generadas == 6.

I16: test_crear_tratamiento_datos_faltantes_retorna_400 [I16] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento_routes.py | **Nivel:** Integración
Objetivo: POST a /api/tratamientos con campos obligatorios omitidos debe responder 400 Bad Request.
PREPARAR (ARRANGE): Payload incompleto con sólo paciente_id.
ACTUAR (ACT): client.post("/api/tratamientos", json=payload).
VERIFICAR (ASSERT): Status HTTP 400 y mensaje de error en JSON.

I17: test_listar_tratamientos_por_paciente_endpoint [I17] IMPLEMENTADA

Archivo: tests/integration/test_tratamiento_routes.py | **Nivel:** Integración
Objetivo: GET a /api/tratamientos/paciente/ retorna 200 OK y la lista de tratamientos del paciente.
PREPARAR (ARRANGE): Tratamiento creado para la paciente en la base de datos.
ACTUAR (ACT): client.get(f"/api/tratamientos/paciente/{paciente_id}").
VERIFICAR (ASSERT): Status HTTP 200, array JSON con longitud 1 y dosis correspondiente.

I18: test_creacion_modelo_registro_toma [I18] IMPLEMENTADA

Archivo: tests/integration/test_registro_toma.py | **Nivel:** Integración
Objetivo: Validar la persistencia de un RegistroToma vinculado a su tratamiento padre.
PREPARAR (ARRANGE): Tratamiento médico persistido en SQLite.
ACTUAR (ACT): Crear y persistir RegistroToma con fecha/hora programada.
VERIFICAR (ASSERT): toma.id is not None, toma.estado == "pendiente" y toma.tratamiento resuelve bien.

I19: test_tomas_generadas_estado_inicial_pendiente [I19] IMPLEMENTADA

Archivo: tests/integration/test_generacion_tomas.py | **Nivel:** Integración
Objetivo: Confirmar que el 100% de las tomas autogeneradas por generar_tomas_tratamiento() nazcan en estado "pendiente".
PREPARAR (ARRANGE): Tratamiento de 2 días cada 8 horas persistido en BD.
ACTUAR (ACT): Invocar generar_tomas_tratamiento(tratamiento.id).
VERIFICAR (ASSERT): Exactamente 6 tomas creadas, todas en estado "pendiente" y fecha_hora_toma is None.

I20: test_transicion_estado_toma_valida [I20] IMPLEMENTADA

Archivo: tests/integration/test_marcar_toma.py | **Nivel:** Integración
Objetivo: toma_service.marcar_toma(...) debe actualizar el estado a "tomada" y asignar timestamp real de consumo.
PREPARAR (ARRANGE): RegistroToma en estado "pendiente".
ACTUAR (ACT): Invocar marcar_toma(toma_id, nuevo_estado="tomada").
VERIFICAR (ASSERT): estado == "tomada", fecha_hora_toma asignada tanto en retorno como en BD.

I21: test_marcar_toma_endpoint_exitoso [I21] IMPLEMENTADA

Archivo: tests/integration/test_toma_routes.py | **Nivel:** Integración
Objetivo: PATCH a /api/tomas/ con nuevo_estado="tomada" debe responder 200 OK y actualizar la BD.
PREPARAR (ARRANGE): Toma en estado pendiente en BD.
ACTUAR (ACT): client.patch(f"/api/tomas/{toma_id}", json={"nuevo_estado": "tomada"}).
VERIFICAR (ASSERT): Status HTTP 200 y JSON confirmando estado "tomada" con timestamp.

I22: test_marcar_toma_estado_invalido_retorna_400 [I22] IMPLEMENTADA

Archivo: tests/integration/test_toma_routes.py | **Nivel:** Integración
Objetivo: PATCH a /api/tomas/ con estado no permitido debe responder 400 Bad Request.
PREPARAR (ARRANGE): Toma existente en BD; payload con estado ficticio "inventado".
ACTUAR (ACT): client.patch(f"/api/tomas/{toma_id}", json={"nuevo_estado": "inventado"}).
VERIFICAR (ASSERT): Status HTTP 400 y mensaje de error descriptivo en JSON.

I23: test_marcar_toma_inexistente_retorna_404 [I23] IMPLEMENTADA

Archivo: tests/integration/test_toma_routes.py | **Nivel:** Integración
Objetivo: PATCH a /api/tomas/99999 (toma inexistente) debe responder código 404 Not Found.
PREPARAR (ARRANGE): ID inexistente 99999.
ACTUAR (ACT): client.patch("/api/tomas/99999", json={"nuevo_estado": "tomada"}).
VERIFICAR (ASSERT): Status HTTP 404 y mensaje de error en JSON.

I24: test_listar_tomas_por_paciente_endpoint [I24] IMPLEMENTADA

Archivo: tests/integration/test_toma_routes.py | **Nivel:** Integración
Objetivo: GET a /api/tomas/paciente/ retorna 200 OK y el listado de 6 tomas programadas.
PREPARAR (ARRANGE): Paciente con prescripción y 6 tomas generadas.
ACTUAR (ACT): client.get(f"/api/tomas/paciente/{paciente_id}").
VERIFICAR (ASSERT): Status HTTP 200 y array JSON con 6 tomas correspondientes.

I25: test_asignar_paciente_a_cuidador_tabla_asociativa [I25] PLANIFICADA

Archivo: tests/integration/test_asignacion_cuidador.py | **Nivel:** Integración
Objetivo: auth_service.asignar_paciente(...) debe persistir la fila en pacientes_cuidadores y rechazar duplicados.
PREPARAR (ARRANGE): 1 usuario cuidador y 1 usuario paciente en BD.
ACTUAR (ACT): Invocar asignar_paciente(cuidador.id, paciente.id) dos veces consecutivas.
VERIFICAR (ASSERT): La primera llamada retorna la asignación; la segunda retorna None evitando duplicidad.

4. Catálogo Detallado de Pruebas End-to-End (Nivel 3 — Flujos Completos de la API)

E1: test_e2e_healthcheck_servidor_y_base_de_datos [E1] PLANIFICADA

Archivo: tests/e2e/test_flujo_clinico_e2e.py | **Nivel:** End-to-End
Objetivo: Peticiones GET a /api/estado y /api/estado-db confirman servidor activo y conexión a BD.
PREPARAR (ARRANGE): Cliente de pruebas levantado.
ACTUAR (ACT): GET a /api/estado y GET a /api/estado-db.
VERIFICAR (ASSERT): Status HTTP 200 en ambos endpoints y respuestas JSON confirmando estado activo.

E2: test_e2e_flujo_registro_login_y_obtencion_jwt [E2] PLANIFICADA

Archivo: tests/e2e/test_flujo_clinico_e2e.py | **Nivel:** End-to-End
Objetivo: Ciclo completo de usuario: registrar cuenta vía HTTP POST y autenticarse para recibir token JWT.
PREPARAR (ARRANGE): Payload con nombre, correo nuevo y contraseña.
ACTUAR (ACT): 1) POST a /api/auth/registro. 2) POST a /api/auth/login.
VERIFICAR (ASSERT): Registro responde 201 Created; Login responde 200 OK y entrega token JWT utilizable.

E3: test_e2e_login_invalido_rechazado_con_401 [E3] PLANIFICADA

Archivo: tests/e2e/test_flujo_clinico_e2e.py | **Nivel:** End-to-End
Objetivo: Validar que una petición de login con contraseña incorrecta sea rechazada con 401 Unauthorized.
PREPARAR (ARRANGE): Usuario registrado en BD; payload con contraseña incorrecta.
ACTUAR (ACT): POST a /api/auth/login con credenciales erradas.
VERIFICAR (ASSERT): Status HTTP 401 Unauthorized y ningún token emitido.

E4: test_e2e_ciclo_completo_prescripcion_y_consumo_tomas [E4] PLANIFICADA

Archivo: tests/e2e/test_flujo_clinico_e2e.py | **Nivel:** End-to-End
Objetivo: Flujo maestro: prescribir tratamiento (3 días cada 12h = 6 tomas), consultar calendario y marcar primera toma.
PREPARAR (ARRANGE): Paciente y medicamento existentes en BD.
ACTUAR (ACT): 1) POST /api/tratamientos. 2) GET /api/tomas/paciente/. 3) PATCH /api/tomas/.
VERIFICAR (ASSERT): Tratamiento 201 con total_tomas_generadas == 6; GET retorna 6 tomas; PATCH responde 200.

E5: test_e2e_auditoria_de_adherencia_paciente [E5] PLANIFICADA

Archivo: tests/e2e/test_flujo_clinico_e2e.py | **Nivel:** End-to-End
Objetivo: Comprobar la auditoría clínica: cuidador consulta tomas y audita fechas reales de toma y notas.
PREPARAR (ARRANGE): Paciente con tomas en estados mixtos (tomada, omitida, retrasada).
ACTUAR (ACT): GET a /api/tomas/paciente/.
VERIFICAR (ASSERT): Status HTTP 200 y cada elemento del reporte contiene fecha_hora_programada y fecha_hora_toma.

5. Comandos Oficiales de Ejecución y Auditoría

Ejecución completa con Pytest:

```
.\venv\Scripts\pytest -v
```

Ejecución con Unittest estándar:

```
.\venv\Scripts\python -m unittest discover -v -s tests -p "test_*.py"
```